Agile compiler project schedule

A proposed 15-week schedule with seven two-week sprints

Comparable scope. Assume 3-4 students contributing approximately 8-10 hours each per week. Build a compiler for a small, statically typed teaching language targeting a stack-based virtual machine with a supplied runtime. Weeks are relative to kickoff. This is a proposed teaching-project plan, not a dated calendar.

Product goal and planning approach

Deliver a usable compiler for integer and boolean expressions, variables, functions, and conditionals, with useful diagnostics and a command-line driver. Each sprint develops a small feature through parsing, semantic checks, code generation, and execution. Tests and documentation evolve with every increment.

The schedule is a forecast. Select detailed sprint work from an ordered backlog according to demonstrated capacity and stakeholder feedback. Keep the product goal and acceptance expectations visible; refine language details as examples expose gaps. Closures, advanced optimization, native code, and a new runtime remain outside the initial scope.

Kickoff and sprint roadmap

When	Sprint goal and work	Working increment / review demo
Week 1	Kickoff: agree on the product goal, initial language examples, core acceptance criteria, team roles, and Definition of Done. Set up version control, automated tests, and a minimal target experiment.	Ordered initial backlog and a tested VM execution path. Sketch the compiler pipeline and interfaces; identify the highest technical risks.
Sprint 1 Weeks 2-3	Compile a minimal integer expression. Build a thin lexer/parser, AST, minimal validation, code emitter, and command-line path.	Compile and execute a source program such as 2 + 3, producing 5. Include a syntax-error example and automated end-to-end test.
Sprint 2 Weeks 4-5	Expand integer expressions: precedence, parentheses, unary signs, subtraction, multiplication, division, and modulo. Agree on arithmetic error behavior.	Execute nested arithmetic correctly; demonstrate negative-number semantics and division-by-zero handling. Keep earlier examples passing.
Sprint 3 Weeks 6-7	Add variables and lexical scope. Extend parsing, symbol tables, type checks, storage allocation, and generated code together.	Execute programs using local bindings. Demonstrate shadowing, rejection of unbound names, and source locations in diagnostics.
Sprint 4 Weeks 8-9	Add booleans, comparisons, and conditionals. Implement condition type checks and branch code generation.	Execute both conditional paths and show that the unselected branch is not executed. Reject invalid condition and operand types.

Complete, inspect, and adapt

When	Sprint goal and work	Working increment / review demo
Sprint 5 Weeks 10-11	Add functions and calls: parameter scope, declared types, argument checking, calling convention, and return values. Use small VM fixtures to resolve call-stack risks.	Compile and run functions and nested calls. Demonstrate errors for invalid arguments and returns. Update user examples and regression tests.
Sprint 6 Weeks 12-13	Complete core acceptance coverage and address the highest-value usability and correctness gaps. Exercise features together; strengthen diagnostics and clean-checkout installation.	Demonstrate the full core language on representative programs. Run the mandatory acceptance suite and publish the remaining defect list.
Sprint 7 Weeks 14-15	Deliver a reliable release: prioritize remaining acceptance failures, documentation, packaging, and handover. Keep capacity for defects and final feedback.	Demonstrate the release from a clean checkout. Deliver tagged source, build instructions, sample programs, automated tests, user guide, and final report.

Cadence within each two-week sprint

Start: hold a 45-60 minute planning session; agree on one sprint goal, acceptance examples, and a realistic set of backlog items. Split implementation tasks so multiple people can contribute to the same feature.

During: use brief daily updates on progress and blockers, asynchronous when class schedules require it. Integrate small changes frequently and run automated tests on every change. Refine the next sprint backlog for about 30 minutes each week.

End: demonstrate executable software in a 30-45 minute review with the instructor/client. Record feedback and reorder the backlog. Hold a 20-30 minute retrospective and choose one concrete process improvement for the next sprint. These meetings fit within the assumed weekly capacity.

Definition of Done for every increment

A feature is done when its agreed acceptance examples pass; code is reviewed and integrated; relevant unit, negative, and end-to-end tests pass with the existing regression suite; diagnostics and user documentation are updated; and it can be demonstrated from a reproducible build. Incomplete work returns to the backlog for replanning.

Ownership, dependencies, and scope decisions

An instructor/client or nominated product owner orders the backlog and clarifies desired behavior. The team chooses implementation tasks and shares responsibility across compiler components; a rotating facilitator tracks blockers and protects the sprint goal. Record brief design decisions for grammar, AST, IR, and calling-convention changes.

Respect technical dependencies while delivering small usable features. Use short, time-boxed experiments for uncertain design choices and pair on high-risk work. Review completed work and remaining effort after each sprint; reserve roughly 15-20% of planned capacity for integration, defects, and uncertainty.

New requests enter the backlog and are prioritized against existing work. If progress threatens week 15, defer optional features first and negotiate any reduction of core scope explicitly. Replan unfinished items instead of treating them as complete. Release only when mandatory acceptance tests pass and no critical correctness defects remain.